1️⃣ Environment & Tool Verification

Verify `kubectl` client version

```
kubectl version --client
```

Output:

```
Client Version: v1.35.0
Kustomize Version: v5.7.1
```

✔ Confirms `kubectl` and `kustomize` are installed correctly.

Verify `kind` installation

```
kind --version
```

Output:

```
kind version 0.31.0
```

✔ Confirms kind is available for local Kubernetes clusters.

2️⃣ Kubernetes Cluster Creation Using kind

kind cluster configuration file

- **API version:** `kind.x-k8s.io/v1alpha4`
- **Topology:**
  - 1 control-plane node
  - 2 worker nodes

Create cluster using config file

```
kind create cluster --name cke --config kind-config.yml
```

✔ Creates a multi-node Kubernetes cluster locally.

Verify cluster nodes

```
kubectl get nodes
```

```
kubectl get nodes -o wide
```

✔ Confirms:

- Control-plane is running
- Worker nodes are ready
- Kubernetes version is v1.35.0

3️⃣ kubectl Context Management

List all contexts

```
kubectl config get-contexts
```

✔ Shows multiple kind contexts:

- `kind-cke`
- `kind-kind-cke1`

Check current context

```
kubectl config current-context
```

Output:

```
kind-cke
```

✔ Confirms all operations are being done on **cke cluster**.

Switch context (when needed)

```
kubectl config use-context kind-cke
```

4️⃣ Understanding Imperative vs Declarative

Imperative

- You tell Kubernetes **WHAT to do right now**
- Example: `kubectl run`

Declarative

- You define **DESIRED STATE in YAML**
- Kubernetes ensures it stays that way

You practiced **both approaches**.

5️⃣ Task 1 – Imperative Pod Creation

Create nginx Pod using imperative command

```
kubectl run nginx-pod --image=nginx
```

✔ Kubernetes creates:

- Pod name: `nginx-pod`
- Container image: `nginx`

Verify Pod

```
kubectl get pods
```

Expected output:

```
nginx-pod   Running
```

✔ Confirms Pod is running successfully.

6️⃣ Exploring Kubernetes API

Inspect Pod API schema

```
kubectl explain pod
```

✔ Learned:

- `apiVersion`
- `kind`
- `metadata`
- `spec`
- `containers`

This helps when **writing YAML manually**.

7️⃣ Task 2 – Generate YAML from Running Pod

Generate YAML from existing Pod

```
kubectl get pod nginx-pod -o yaml > nginx-pod.yml
```

✔ Converts **live Pod object** into YAML.

Edit the YAML

```
vim nginx-pod.yml
```

Remove auto-generated fields (VERY IMPORTANT)

Removed:

```
status:
creationTimestamp:
resourceVersion:
uid:
managedFields:
```

❗ These fields are **managed by Kubernetes**, not allowed when creating new objects.

Update Pod name

```
metadata:
  name: nginx-new
```

Final cleaned YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-new
spec:
  containers:
  - name: nginx
    image: nginx
```

Create new Pod using YAML

```
kubectl apply -f nginx-pod.yml
```

Verify Pods

```
kubectl get pods
```

Expected:

```
nginx-pod
nginx-new
```

✔ Demonstrates **imperative → declarative transition**.

8️⃣ Declarative Pod Creation (Manual YAML)

Create YAML file

```
vim pod.yml
```

Example:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-declarative
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

Apply YAML

```
kubectl create -f pod.yml
```

Verify

```
kubectl get pods
```

9️⃣ Editing Pods & Dry-Run Practice

Edit running Pod

```
kubectl edit pod nginx-pod
```

✔ Opens Pod spec directly from API server.

Dry-run YAML generation (WRONG)

```
kubectl run nginx --image=nginx --dry-run=client -o yml > newpod.yml
```

❌ Error:

```
unknown output format "yml"
```

Dry-run YAML generation (CORRECT)

```
kubectl run nginx --image=nginx --dry-run=client -o yaml > newpod.yaml
```

✔ Generates YAML **without creating resource**.

🔟 Task 3 – Troubleshooting a Broken YAML

Faulty YAML

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: test
  name: redis
spec:
  containers:
  - image: rediss
    name: redis
```

Save file

```
vim redis.yml
```

Apply YAML

```
kubectl apply -f redis.yml
```

Error observed

```
Error from server (BadRequest):
image "rediss" not found
```

Identify issue

❌ Image name is wrong
✅ Correct image is `redis`

Fix YAML

```
vim redis.yml
```

Corrected:

```
image: redis
```

Apply fixed YAML

```
kubectl apply -f redis.yml
```

Verify

```
kubectl get pods
```

Expected:

```
redis        Running
nginx-pod   Running
nginx-new   Running
```

1️⃣1️⃣ Pod Inspection & Debugging

Describe Pod

```
kubectl describe pod nginx-pod
```

✔ Shows:

- Events
- Image pull status
- Node assignment

Show labels

```
kubectl get pods --show-labels
```

Wide output

```
kubectl get pods -o wide
```

✔ Shows:

- Pod IP
- Node name

1️⃣2️⃣ Final Cluster Verification

```
kubectl get pods
kubectl get nodes
kubectl cluster-info
```

QA

1️⃣ Question

Is `kubectl run nginx-pod --image=nginx` imperative or declarative?

✅ **Answer:**

`imperative`

2️⃣ Question

Command to generate YAML from an existing running Pod (`nginx-pod`)?

✅ **Answer:**

`kubectl get pod nginx-pod -o yaml > nginx-pod.yml`

3️⃣ Question

Command to create a new Pod from a YAML file?

✅ **Answer:**

```
kubectl apply -f nginx-new.yml
```

4️⃣ Question

Command to verify that Pods are running?

✅ **Answer:**

```
kubectl get pods
```

5️⃣ Question

Command to see full Pod details and events?

✅ **Answer:**

```
kubectl describe pod nginx-pod
```

6️⃣ Question

Command used to apply the faulty Redis YAML and see the error?

✅ **Answer:**

```
kubectl apply -f redis.yml
```

7️⃣ Question

Command to verify Redis Pod is running after fixing the error?

✅ **Answer:**

```
kubectl get pods
```

8️⃣ Question

Command to check the current kubectl context?

✅ **Answer:**

```
kubectl config current-context
```

9️⃣ Question

Command to list all kubectl contexts?

✅ **Answer:**

```
kubectl config get-contexts
```

🔟 Question

Command to switch to `kind-cke` context?

✅ **Answer:**

```
kubectl config use-context kind-cke
```

1️⃣1️⃣ Question

Command to inspect the Pod API schema?

✅ **Answer:**

```
kubectl explain pod
```

1️⃣2️⃣ Question

Command to edit a running Pod directly?

✅ **Answer:**

```
kubectl edit pod <pod-name>
```

1️⃣3️⃣ Question

Command to generate YAML using client-side dry run?

✅ **Answer:**

```
kubectl run pod-nginx --image=nginx --dry-run=client -o yaml >
new.yaml
```

1️⃣4️⃣ Question

Command to view Pods along with labels?

✅ **Answer:**

```
kubectl get pods --show-labels
```

1️⃣5️⃣ Question

Command to view Pods with node name and Pod IP?

✅ **Answer:**

```
kubectl get pods -o wide
```

1️⃣6️⃣ Question

Command to list all nodes in the cluster?

✅ **Answer:**

```
kubectl get nodes
```

1️⃣7️⃣ Question

Command to list nodes with detailed information?

✅ **Answer:**

```
kubectl get nodes -o wide
```

1️⃣8️⃣ Question

Command to check cluster information?

✅ **Answer:**

```
kubectl cluster-info
```

1️⃣9️⃣ Question

Command to create a kind cluster using a config file?

✅ **Answer:**

```
kind create cluster --name cke --config kind-config.yml
```

2️⃣0️⃣ Question

Command to check kubectl client version?

✅ **Answer:**

```
kubectl version --client
```

Question in your head	Command
Is it running?	`kubectl get pods`
Where is it running?	`kubectl get pods -o wide`
Why is it failing?	`kubectl describe pod`
What config is applied?	`kubectl get pod -o yaml`

1️⃣ ReplicationController Practice (Detailed)

What you did (ReplicationController is used to ensure *N number of Pods are always running* in a Kubernetes cluster.)

You created a ReplicationController (RC) to ensure multiple identical Pods are always running.

YAML used (`rc.yml`)

- **kind:** `ReplicationController`
- **replicas:** `3`

selector:
`app: nginx`

- **Pod template runs nginx container on port 80**

Commands executed

```
kubectl apply -f rc.yml
kubectl get rc
kubectl scale rc nginx-rc --replicas=10 # scale anytime
kubectl describe pod <pod-name>
```

What happened internally

- **Kubernetes created 3 nginx Pods**
- **If any Pod is deleted, RC automatically recreates it**
- **RC continuously compares:**

  `desired replicas (3) == actual replicas`

2️⃣ ReplicaSet Practice (Detailed)

Why you moved to ReplicaSet

- ReplicationController is legacy
- ReplicaSet is the modern replacement
- ReplicaSet supports advanced selectors

Conversion: RC → RS

YAML used (`rs.yml`)

```yaml
apiVersion: apps/v1
kind: ReplicaSet
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
```

Commands executed

```
kubectl delete rc nginx-rc
kubectl apply -f rs.yml
kubectl get rs
kubectl get pods
```

What happened internally

- RC stopped managing Pods
- ReplicaSet took over Pod management
- New Pods were created by ReplicaSet

Scaling ReplicaSet

Command

```
kubectl scale rs nginx-rs --replicas=10 # scale anytime
```

Result

- ReplicaSet created 10 nginx Pods
- Pods were distributed across nodes
- Kubernetes scheduler handled placement automatically

Verification

```
kubectl get pods
```

3️⃣ Deployment Concept & Practice (Detailed)

What you learned

Deployment manages ReplicaSets and provides scaling, rolling updates, and rollback updates.

Deployment is a higher-level controller that manages:

```
Deployment → ReplicaSet → Pods
```

Why Deployment is important

- Rolling updates
- Rollbacks
- Version history

Deployment creation

YAML (deployment.yml)

- kind: Deployment
- replicas: 3
- image: nginx
- containerPort: 80

Command

```
kubectl apply -f deployment.yml
```

Verification

```
kubectl get deployments
kubectl get rs
kubectl get pods
kubectl get all
```

What you observed

- Deployment created a ReplicaSet
- ReplicaSet created 3 Pods
- Everything is tracked and versioned

4️⃣ Deployment Image Update (Rollout) — Detailed

Command you ran

```
kubectl set image deploy/nginx-deploy nginx=nginx:1.9.1
```

What happened internally

1. Deployment detected spec change
2. Created a new ReplicaSet
3. Gradually:
    - Created Pods with new image
    - Deleted old Pods (rolling update)
4. Stored revision history

Verification commands

```
kubectl rollout status deployment/nginx-deploy
```

```
kubectl rollout history deployment/nginx-deploy
```

(You may or may not have run these, but the rollout still occurred.)

5️⃣ Deployment YAML via Dry-Run (Detailed)

Purpose

- Generate clean, reusable YAML
- No resource created immediately

Command

```
kubectl create deploy nginx-new --image —-dry-run=client -o yaml >
deployment.yml
```

Result

- `deploy.yaml` contains:
  - apiVersion: `apps/v1`
  - kind: `Deployment`
  - replicas: `1` (default)
  - nginx container definition

1 Question

Which command did you use to create the ReplicationController from a YAML file?

Answer

```
kubectl apply -f rc.yml
```

2 Question

Which command shows the status of ReplicationControllers?

Answer

```
kubectl get rc
```

3 Question

Which command shows all Pods created by the ReplicationController?

Answer

```
kubectl get pods
```

4 Question

Which command did you use to inspect full details of a Pod created by the RC?

Answer

```
kubectl describe pod <pod-name>
```

5 Question

Which command deletes the ReplicationController named `nginx-rc`?

Answer

```
kubectl delete rc/nginx-rc
```

6️⃣ Question

Which command did you use to create the ReplicaSet from `rs.yml`?

Answer

```
kubectl apply -f rs.yml
```

7️⃣ Question

Which command lists ReplicaSets in the cluster?

Answer

```
kubectl get rs
```

8️⃣ Question

Which command scales the ReplicaSet `nginx-rs` to 10 replicas?

Answer

```
kubectl scale rs nginx-rs --replicas=10
```

9️⃣ Question

Which command verifies that 10 Pods were created by the ReplicaSet?

Answer

```
kubectl get pods
```

🔟 Question

Which command did you use to edit the ReplicaSet configuration directly?

Answer

```
kubectl edit rs nginx-rs
```

1️⃣1️⃣ Question

Which command did you use to create the Deployment from a YAML file?

Answer

```
kubectl apply -f deployment.yml
```

1️⃣2️⃣ Question

Which command shows all Deployments?

Answer

```
kubectl get deployments
```

1️⃣3️⃣ Question

Which command shows the full hierarchy (Deployment, ReplicaSet, Pods)?

Answer

```
kubectl get all
```

1️⃣4️⃣ Question

Which command did you use to update the nginx image in the Deployment?

Answer

```
kubectl set image deploy/nginx-deploy nginx=nginx:1.9.1
Or
Kubectl edit deployment nginx-deploy # edit direct yml during live pod
running
```

1️⃣5️⃣ Question

What happens internally when `kubectl set image` is executed on a Deployment?

Answer

```
A new ReplicaSet is created and a rolling update starts
```

16 Question

Which command shows rollout history of a Deployment?

Answer

```
kubectl rollout history deployment nginx-deploy
```

17 Question

Which command generates a Deployment YAML without creating the resource?

Answer

```
kubectl create deploy nginx-new --image=nginx --dry-run=client -o yaml
> deploy.yaml
```

18 Question

Which Kubernetes object directly manages Pods in a Deployment?

Answer

```
ReplicaSet
```

19 Question

Which object supports rollback functionality?

Answer

```
Deployment
```

20 Question

What is the correct hierarchy you practiced?

Answer

```
Deployment → ReplicaSet → Pods
```

TASK 1️⃣ Create Deployment (nginx, 3 replicas)

🔹 Requirements

- **Deployment name:** `nginx`
- **Replicas:** `3`
- **Image:** `nginx:1.23.0`
- **Deployment label:** `tier=backend`
- **Pod label:** `app=v1`

✅ Step 1: Generate Deployment YAML (Dry-Run)

```
kubectl create deployment nginx --image=nginx:1.23.0 --dry-run=client
-o yaml > nginx-deploy.yml
```

✅ Step 2: Edit YAML

```
vim nginx-deploy.yml
```

✅ Correct YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  labels:
    tier: backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: v1
  template:
    metadata:
      labels:
```

```
        app: v1
    spec:
      containers:
      - name: nginx
        image: nginx:1.23.0
```

✅ Step 3: Apply Deployment

```
kubectl apply -f nginx-deploy.yml
```

✅ Step 4: Verify Deployment & Pods

```
kubectl get deployment nginx
kubectl get pods
```

Expected:

```
3/3 replicas running
```

Update Image to nginx:1.23.4

✅ Step 5: Update Image

```
kubectl set image deployment/nginx nginx=nginx:1.23.4
```

✅ Step 6: Verify Rollout

```
kubectl rollout status deployment/nginx
```

✅ Step 7: Verify Image in Pods

```
kubectl describe deployment nginx| grep -i image
```

Expected:

```
Image: nginx:1.23.4
```

PART 3️⃣ Add Change Cause

✅ Step 8: Annotate Change Cause

```
 annotate deployment/nginx \
kubernetes.io/change-cause="Pickkubectl up patch version"
```

PART 4️⃣ Scale Deployment to 5 Replicas

✅ Step 9: Scale

```
kubectl scale deployment nginx --replicas=5
```

✅ Step 10: Verify Scaling

```
kubectl get deployment nginx
kubectl get pods
```

Expected:

```
5/5 replicas running
```

PART 5️⃣ View Rollout History

✅ Step 11: Rollout History

```
kubectl rollout history deployment/nginx
```

PART 6️⃣ Rollback to Revision 1

✅ Step 12: Rollback

```
kubectl rollout undo deployment/nginx --to-revision=1
```

✅ Step 13: Verify Rollback

```
kubectl rollout status deployment/nginx
kubectl describe pod <pod-name> | grep Image
```

Expected:

```
Image: nginx:1.23.0
```

🚨 PART 7️⃣ TROUBLESHOOTING YAML – ISSUE 1

❌ GIVEN (WRONG)

```
apiVersion: v1
kind: Deployment
```

❌ Errors

- Wrong `apiVersion`
- `Deployment` must use `apps/v1`

✅ FIXED YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
  labels:
    env: demo
spec:
  replicas: 3
  selector:
    matchLabels:
      env: demo
  template:
    metadata:
      labels:
        env: demo
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
```

✅ Apply Fixed YAML

```
kubectl apply -f fixed-deploy.yml
```

🚨 PART 8️⃣ TROUBLESHOOTING YAML – ISSUE 2

❌ GIVEN (WRONG SELECTOR)

```
selector:
  matchLabels:
    env: dev
```

❌ Error

- Selector (`env=dev`)
- Pod label (`env=demo`)
  ❌ Pods will NOT be created

✅ FIXED YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
  labels:
    env: demo
spec:
  replicas: 3
  selector:
    matchLabels:
      env: demo
  template:
    metadata:
      labels:
        env: demo
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
```

✅ Apply Fixed YAML

```
kubectl apply -f fixed-deploy.yml
```

✅ FINAL VERIFICATION

```
kubectl get deployments
kubectl get rs
kubectl get pods
```

1️⃣ What is NodePort?

NodePort is a Service type used to access applications from outside the Kubernetes cluster.

- **Opens a port on every node**
- **Traffic sent to `NodeIP:NodePort` reaches the application Pods**

2️⃣ Why NodePort is needed

- **Pod IPs are temporary**
- **Pods can be deleted and recreated**
- **We need a stable way to access Pods externally**

👉 **NodePort provides a fixed external entry point**

3️⃣ Ports Explained (VERY IMPORTANT)

🔹 targetPort

- **Port where the application runs inside the container**
- **Example: nginx runs on port `80`**

```
targetPort: 80
```

🔹 Service port (`port`)

- **Internal Service port**
- **Used for Pod-to-Pod communication inside the cluster**

`port: 80`

Access inside cluster:

`http://service-name:80`

🔹 NodePort

- **External port opened on all nodes**
- **Used by users / browser**

Example:

`NodeIP:32209`

4️⃣ Port Relationship

nginx Copy code

```
NodePort → Service port → targetPort
```

Level	Port
Node	NodePort
Service	port
Pod	targetPort

5️⃣ Prerequisites Before Creating a Service

✅ Kubernetes cluster must exist

- Control-plane + worker nodes

✅ Pods must already be running

- Usually created using a Deployment
- Pods must have labels

Example label:

```
app: v1
```

⚠️ **Service does NOT create Pods**

6️⃣ What I Did (Step-by-Step)

Step 1: Created Deployment

- Deployed nginx
- Created 3 Pods
- Pods were running

```
kubectl get pods
```

```
dev@dev-ThinkCentre-M700:~$ cat svc.yml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: NodePort
  selector:
    app: v1
  ports:
  - port: 80        # Service port
    targetPort: 80  # Pod/container port
```

Step 2: Created NodePort Service

- Applied Service YAML
- Service selected existing Pods using labels
- Kubernetes auto-assigned NodePort

```
kubectl apply -f svc.yml
```

Step 3: Verified Service

```
kubectl get svc
```

Example output:

```
80:32209/TCP
```

7️⃣ How I Accessed the Application

🔹 **External access (browser / curl)**

```
http://<control-plane-ip>:<nodeport>
```

Example:

```
http://172.18.0.2:32209
```

🔹 **Internal access (inside cluster)**

```
http://service-name:80
```

8️⃣ Traffic Flow

External

```
Browser

 → NodeIP:NodePort

 → Service port

 → Pod targetPort
```

Internal

```
Pod
 → Service (ClusterIP:port)
 → Pod
```

9️⃣ Key Takeaways

- **Deployment → creates Pods**
- **Service → exposes Pods**
- **ClusterIP → internal access only**
- **NodePort → internal + external access**
- **NodePort is optional to define (Kubernetes can auto-assign)**

Q1️⃣ What is a Service (svc) in Kubernetes?

Answer:
A Service provides a stable way to access Pods using a fixed IP and DNS name.

Q2️⃣ Why do we need a Service?

Answer:
Because Pods are temporary and their IPs change when they restart.

Q3️⃣ What is NodePort?

Answer:
NodePort is a Service type that exposes Pods externally by opening a port on every node.

Q4 What is Service port (`port`)?

Answer:
Service port is the port on which the Service listens inside the cluster.

Q5 What is targetPort?

Answer:
targetPort is the port on the container (Pod) where the application is running.

Q6 What is the relationship between NodePort, port, and targetPort?

Answer:

```
NodePort → Service port → targetPort
```

Q7 Is it mandatory to specify nodePort in the Service YAML?

Answer:
No. Kubernetes can automatically assign a NodePort.

Q8 What is the default NodePort range?

Answer:
`30000–32767`

Q9 Can a Service work without Pods?

Answer:
No. Service requires running Pods with matching labels.

Q10 How does a Service select Pods?

Answer:
Using label selectors.

Q11 What happens if Service selector does not match any Pod?

Answer:
Service is created, but traffic does not reach any Pod (no endpoints).

Q12 How to check Service endpoints?

```
kubectl get endpoints <service-name>
```

Q13 How to create a NodePort Service using YAML?

```
apiVersion: v1

kind: Service

metadata:

  name: nginx-svc

spec:

  type: NodePort

  selector:

    app: v1

  ports:

  - port: 80

    targetPort: 80
```

Q14 How to create a NodePort Service using a command?

```
kubectl expose deployment nginx \

  --type=NodePort \

  --port=80
```

Q15 How to generate Service YAML using dry-run?

```
kubectl expose deployment nginx \
```

```
  --type=NodePort \
  --port=80 \
  --dry-run=client -o yaml > svc.yml
```

Q16 How to view Service details?

```
kubectl get svc
```

Q17 How to describe a Service?

```
kubectl describe svc nginx-svc
```

Q18 How to access a NodePort Service externally?

Answer:

```
http://NodeIP:NodePort
```

Q19 How to access a Service internally?

Answer:

```
http://service-name:port
```

Q20 Does NodePort open on one node or all nodes?

Answer:
NodePort opens the same port on all nodes.

1 ClusterIP Service

What is ClusterIP?

- Default Service type
- Used for internal communication only
- Not accessible from outside the cluster

Why ClusterIP is used

- Secure internal access
- Stable IP and DNS
- Decouples Pods from clients

How ClusterIP works

`Pod → ClusterIP Service → Pod`

- Service gets a virtual IP (e.g. `10.96.x.x`)
- Service load-balances traffic across Pods

Example use cases

- Frontend → Backend
- Backend → Database
- Internal microservice communication

Access method

`http://service-name:port`

ClusterIP YAML example

```
apiVersion: v1

kind: Service

metadata:

  name: backend-svc

spec:

  type: ClusterIP

  selector:
```

```
    app: backend
  ports:
  - port: 80
    targetPort: 80
```

2️⃣ LoadBalancer Service

What is LoadBalancer?

- Service type for external user access
- Used mainly in cloud environments
- Automatically provisions a cloud load balancer

Why LoadBalancer is used

- Production-ready external access
- Provides public IP / DNS
- High availability and scalability

How LoadBalancer works

```
Users
 → LoadBalancer
 → ClusterIP Service
 → Pods
```

Internally:

- **LoadBalancer still uses ClusterIP**
- **Traffic reaches Pods through Service**

Example use cases

- **Public web applications**
- **APIs exposed to users**
- **Frontend applications**

Access method

`http://<external-ip>` **or**

`http://myapp.com`

LoadBalancer YAML example

```
apiVersion: v1

kind: Service

metadata:

  name: frontend-lb

spec:

  type: LoadBalancer

  selector:

    app: frontend

  ports:

  - port: 80

    targetPort: 80
```

Q1 What is a ClusterIP Service?

Answer:
ClusterIP is a Service type used for internal communication only inside the Kubernetes cluster.

Q2 When should we use ClusterIP?

Answer:
Use ClusterIP when the application should be accessed only by other Pods, not by external users.

Q3 How do Pods access a ClusterIP Service?

Answer:
Using the Service DNS name and port.

```
http://service-name:port
```

Q4 Give an example of ClusterIP Service YAML.

```
apiVersion: v1

kind: Service

metadata:

  name: backend-svc

spec:

  type: ClusterIP

  selector:

    app: backend

  ports:

  - port: 80

    targetPort: 80
```

Q5 How to check ClusterIP of a Service?

```
kubectl get svc
```

Q6️⃣ Can ClusterIP be accessed from a browser?

Answer:
❌ No. ClusterIP works only inside the cluster.

Q7️⃣ What is a LoadBalancer Service?

Answer:
LoadBalancer is a Service type that exposes applications externally using a cloud-managed load balancer.

Q8️⃣ When should we use LoadBalancer?

Answer:
Use LoadBalancer when you want to expose an application to external users (production apps).

Q9️⃣ How do users access a LoadBalancer Service?

Answer:
Using the external IP or DNS provided by the cloud.

```
http://<external-ip>
```

Q🔟 Give an example of LoadBalancer Service YAML.

```
apiVersion: v1

kind: Service

metadata:

  name: frontend-lb

spec:

  type: LoadBalancer

  selector:
```

```
    app: frontend
  ports:
  - port: 80
    targetPort: 80
```

Q11 How to check external IP of a LoadBalancer Service?

```
kubectl get svc
```

Q12 What happens if LoadBalancer is used on Kind or Minikube?

Answer:
`EXTERNAL-IP` **may stay** `<pending>` **because no real cloud load balancer exists.**

Q13 Does LoadBalancer internally use ClusterIP?

Answer:
✅ **Yes. LoadBalancer always creates a ClusterIP internally.**

Q14 What is the traffic flow for LoadBalancer?

Answer:

```
User → LoadBalancer → ClusterIP Service → Pod
```

Q15 How does a Service know which Pods to send traffic to?

Answer:
Using label selectors.

Q16 How to check which Pods are connected to a Service?

```
kubectl get endpoints <service-name>
```

Q1⃣7⃣ What happens if Service selector labels are wrong?

Answer:
Service is created, but no traffic reaches Pods.

Q1⃣8⃣ Can Service create Pods?

Answer:
❌ **No. Service only exposes existing Pods.**

Q1⃣9⃣ Key difference between ClusterIP and LoadBalancer?

Feature	ClusterIP	LoadBalancer
Access	Internal only	External
Browser access	× No	✅ Yes
Use case	Backend services	Frontend apps
Cloud needed	No	Yes

1⃣ What is a Namespace?

- **A Namespace is a logical partition (folder) inside a Kubernetes cluster.**
- **It helps organize, isolate, and manage Kubernetes resources.**

👉 **One cluster can have multiple namespaces.**

2⃣ Why do we need Namespace?

✔ **Resource Isolation**

- **Separate workloads like:**
  - **dev**
  - **test**
  - **prod**

✔ Avoid Name Conflicts

- **Same resource name can exist in different namespaces.**

Example:

`default/nginx`

`demo/nginx`

✔ Security (RBAC)

- **Control who can access what namespace.**

✔ Resource Control

- **Apply CPU & memory limits using:**
  - **ResourceQuota**
  - **LimitRange**

3️⃣ Default Namespaces (Important)

Check namespaces:

bash Copy code

```bash
kubectl get ns
```

Common namespaces:

Namespace	Purpose
default	User workloads
kube-system	Kubernetes components
kube-public	Public data
kube-node-lease	Node heartbeats

4️⃣ Creating a Namespace

Method 1: Using command

```
kubectl create namespace demo
```

Method 2: Using YAML

```
apiVersion: v1
kind: Namespace
metadata:
  name: demo
```

Apply:

```
kubectl apply -f ns.yml
```

5 Creating Resources in a Namespace

Using command flag

```
kubectl apply -f deploy.yml -n demo
```

Or inside YAML

```
metadata:
  name: nginx
  namespace: demo
```

6 Viewing Resources in Namespace

```
kubectl get pods -n demo
kubectl get svc -n demo
kubectl get deploy -n demo
```

All namespaces:

```
kubectl get pods -A
```

7 Switching Default Namespace (Optional)

```
kubectl config set-context --current --namespace=demo
```

Check:

```
kubectl config view --minify | grep namespace
```

8️⃣ Namespace and Service Communication (VERY IMPORTANT)

Same Namespace

```
curl http://service-name
```

Different Namespace

```
curl http://service-name.namespace
```

Full DNS (FQDN)

```
curl http://service-name.namespace.svc.cluster.local
```

9️⃣ Namespace Scope vs Cluster Scope

Namespace-scoped resources

- **Pod**
- **Service**
- **Deployment**
- **ConfigMap**
- **Secret**

Cluster-scoped resources

- **Node**
- **Namespace**
- **PersistentVolume**
- **ClusterRole**

🔟 Deleting a Namespace

⚠️ **Deletes everything inside it**

```
kubectl delete namespace demo
```

⭐ Common Mistakes

❌ **Forgetting `-n namespace`**
❌ **Assuming services work across namespaces without DNS**
❌ **Thinking namespace gives network isolation (it does NOT)**

🧠 One-Line Exam Answers

- **Namespace is used to logically isolate and organize resources in a Kubernetes cluster.**
- **Resources in different namespaces communicate using service-name.namespace.**

🧾 Quick Command Cheat Sheet

```
kubectl get ns
kubectl create ns demo
kubectl get pods -n demo
kubectl apply -f file.yml -n demo
kubectl delete ns demo
```

🔹 Task Title

Understanding Cross-Namespace Pod and Service Communication in Kubernetes

🔹 Objective (Purpose of the Task)

To understand how Kubernetes networking works across namespaces, specifically:

- **Pod-to-Pod communication across namespaces**
- **Service-to-Service communication across namespaces**
- **Namespace-scoped DNS behavior**
- **Importance of using Service FQDNs for cross-namespace access**

🔹 Prerequisites

- **Running Kubernetes cluster**
- **`kubectl` configured and working**
- **Basic understanding of:**
  - **Pods**
  - **Deployments**
  - **Services**
  - **Namespaces**

🔹 Task Steps

Step 1: Create Namespaces

Create two namespaces:

- `ns1`
- `ns2`

```
kubectl create namespace ns1
kubectl create namespace ns2
```

Verify:

```
kubectl get namespaces
```

Step 2: Create Deployments in Each Namespace

Create an nginx Deployment with 1 replica in each namespace.

```
kubectl create deployment deploy-ns1 --image=nginx -n ns1
kubectl create deployment deploy-ns2 --image=nginx -n ns2
```

Verify Pods:

```
kubectl get pods -n ns1 -o wide
kubectl get pods -n ns2 -o wide
```

Step 3: Test Pod-to-Pod Communication Across Namespaces

1. **Note the Pod IP of the Pod running in `ns2`**
2. **Exec into the Pod running in `ns1`**

```
kubectl exec -it deploy/deploy-ns1 -n ns1 -- bash
```

3. **Curl the Pod IP of `ns2`**

```
curl <ns2-pod-ip>
```

✅ **Expected Result: nginx welcome page is returned.**

Step 4: Scale Both Deployments

Scale both Deployments from 1 to 3 replicas.

```
kubectl scale deployment deploy-ns1 --replicas=3 -n ns1
kubectl scale deployment deploy-ns2 --replicas=3 -n ns2
```

Verify:

```
kubectl get pods -n ns1
kubectl get pods -n ns2
```

Step 5: Create ClusterIP Services for Each Deployment

Create Services to expose both Deployments.

Service for `ns1`

```
apiVersion: v1
kind: Service
metadata:
  name: svc-ns1
  namespace: ns1
spec:
  selector:
    app: deploy-ns1
  ports:
  - port: 80
    targetPort: 80
```

Service for `ns2`

```
apiVersion: v1
kind: Service
metadata:
  name: svc-ns2
  namespace: ns2
spec:
  selector:
    app: deploy-ns2
  ports:
  - port: 80
    targetPort: 80
```

Apply:

```
kubectl apply -f svc-ns1.yaml
kubectl apply -f svc-ns2.yaml
```

Verify:

```
kubectl get svc -n ns1
kubectl get svc -n ns2
```

Step 6: Test Service IP Communication Across Namespaces

1. **Get the ClusterIP of `svc-ns2`**

2. **Exec into a Pod in `ns1`**

```
kubectl exec -it deploy/deploy-ns1 -n ns1 -- bash
```

3. **Curl the Service IP**

```
curl <svc-ns2-cluster-ip>
```

✅ **Expected Result: nginx welcome page is returned.**

Step 7: Test Service Name Resolution (Failure Case)

From inside the `ns1` Pod:

```
curl svc-ns2
```

❌ **Expected Result: Host not found / DNS resolution failure**

Step 8: Test Service Access Using FQDN

```
curl svc-ns2.ns2
```

or

```
curl svc-ns2.ns2.svc.cluster.local
```

✅ **Expected Result: nginx welcome page is returned.**

Step 9: Cleanup

Delete both namespaces:

```
kubectl delete namespace ns1
kubectl delete namespace ns2
```

🔹 Observations

- **Pod IP communication works across namespaces**
- **Service IP communication works across namespaces**
- **Short service names are namespace-scoped**
- **Cross-namespace service access requires FQDN**